COMP132: Advanced Programming

Programming Project Report

Kulatro Game

Berat Yucal, 86754

Spring 2026

Part 1

General Demo Information:

Kulatro is a Java Swing card game where the player logs in, chooses a deck and difficulty, plays four rounds, uses special cards, and tries to beat the average target score. I built the interface with Swing frames, buttons, labels, dialogs, panels, and mouse listeners [2], [3]. The card and background visuals were generated with AI assistance and then added to the Java project as image resources [6], [7].

1. Game Initialization from GUI

The game starts with the login and registration screen shown in Figure 1. A player can create an account or enter existing credentials. Registered usernames are stored in players.txt, which is shown later in Figure 16. After login, the player reaches the main menu in Figure 2. This menu contains New Game, Load Game, Select Deck, Logout, and the leaderboard panel. To start a session, the game asks for a session name as shown in Figure 3, then lets the player choose the deck and difficulty settings.

Difficulty changes the round targets. Easy uses 40, 55, 70, and 85; Medium uses 50, 65, 80, and 95; Hard uses 60, 75, 90, and 105. Deck choice changes card types, card art, special cards, and strategy. The rules window in Figure 4 explains the overall goal and gameplay flow.

Figure 1. Login screen of Kulatro, showing the username and password fields together with Login, Register, and How to Play buttons.

Figure 2. Main menu after login, showing user berat, total score, New Game, Load Game, Select Deck, Logout, and the leaderboard table.

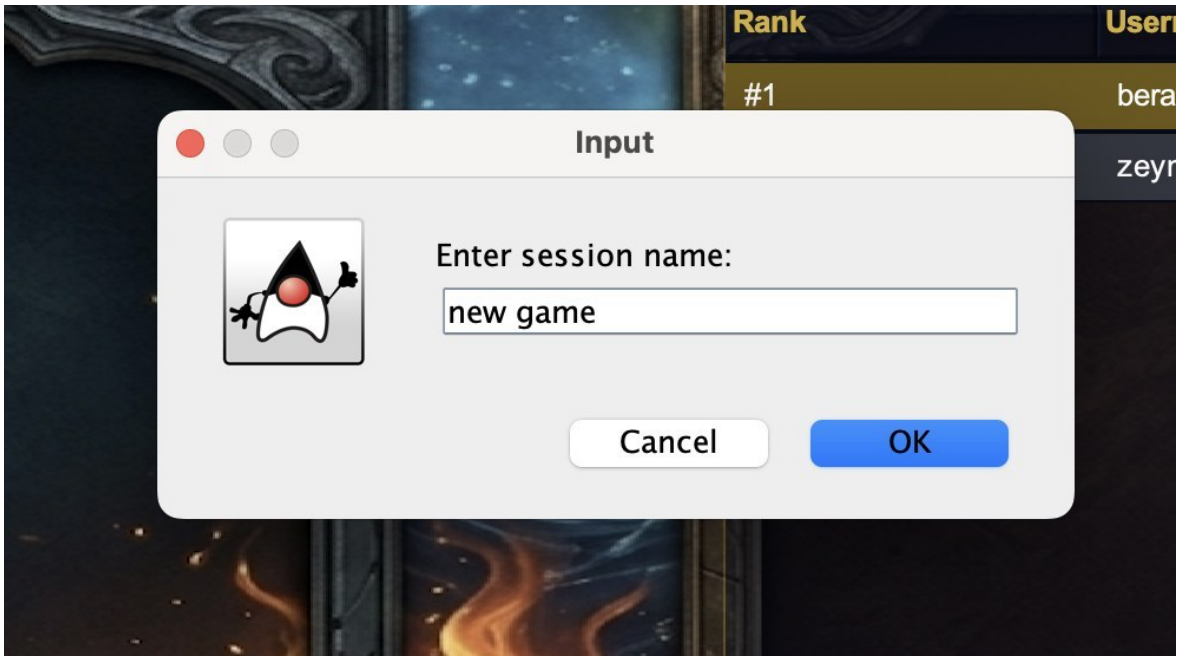

Figure 3. Session-name input dialog that appears when the player starts a new game from the main menu.

Figure 4. How to Play window on the Overview tab, explaining the four-round goal and final average-based win condition.

2. Differences Between Deck Types

Kulatro has three decks. Alchemy uses Fire, Water, Earth, and Air. Element uses Hydrogen, Oxygen, Nitrogen, and Carbon Dioxide. Quantum uses Quark, Boson, Gluon, and Photon. Figures 5, 6, and 7 show the visual difference between these decks in active gameplay. The HUD layout stays the same, but card artwork, type labels, special-card text, and deck identity change.

Deck	Types	Special cards	Strategy
Alchemy	Fire, Water, Earth, Air	Philosopher's Stone, Transmutation, Elemental Fusion, Catalyst	Direct score multipliers and type-based scoring.
Element	Hydrogen, Oxygen, Nitrogen, Carbon Dioxide	Periodic Boost, Noble Gas, Isotope Decay, Electron Bond	Value changes, hand reset, and artificial pair creation.
Quantum	Quark, Boson, Gluon, Photon	Quantum Entanglement, Superposition, Gluon Bind, Photon Burst	Single-card amplification, best-pattern play, merging, and deck preview.

Figure 5. Alchemy gameplay screen with Air-4, Air-1, Earth-1, and Earth-5 in hand, while Elemental Fusion is shown in the side panel.

Figure 6. Element gameplay screen with Oxygen, Carbon Dioxide, and Nitrogen cards in hand, while Isotope Decay is shown as the selected special card.

Figure 7. Quantum gameplay screen with Gluon, Quark, and Photon cards in hand, while Quantum Entanglement is shown in the side panel.

3. Gameplay Demonstration

3.a. Card Interaction

During a round, GameFrame displays four cards in the hand area and tracks selected cards in the side panel. The player selects or deselects cards with mouse clicks. Selected cards are used for Submit, Discard, or special-card actions. The current round, target, score, remaining discards, deck count, and deck type remain visible in the top HUD.

3.b. Discard Mechanism

The discard system only allows numeric cards to be discarded. If a player presses Discard without selecting a valid card, the interface shows the message in Figure 8. A valid discard removes the card from the hand, adds it to the discard pile, decreases the discard counter, and draws a replacement card from the deck.

```
public boolean discardFromHand(Card card) {
    if (card instanceof SpecialCard) return false;
    if (!hasDiscardAvailable()) return false;
    if (!hand.contains(card)) return false;
    hand.remove(card);
    discardPile.add(card);
```

```
    remainingDiscards--;
    roundDiscards++;
    drawOneCard();
    return true;
}
```

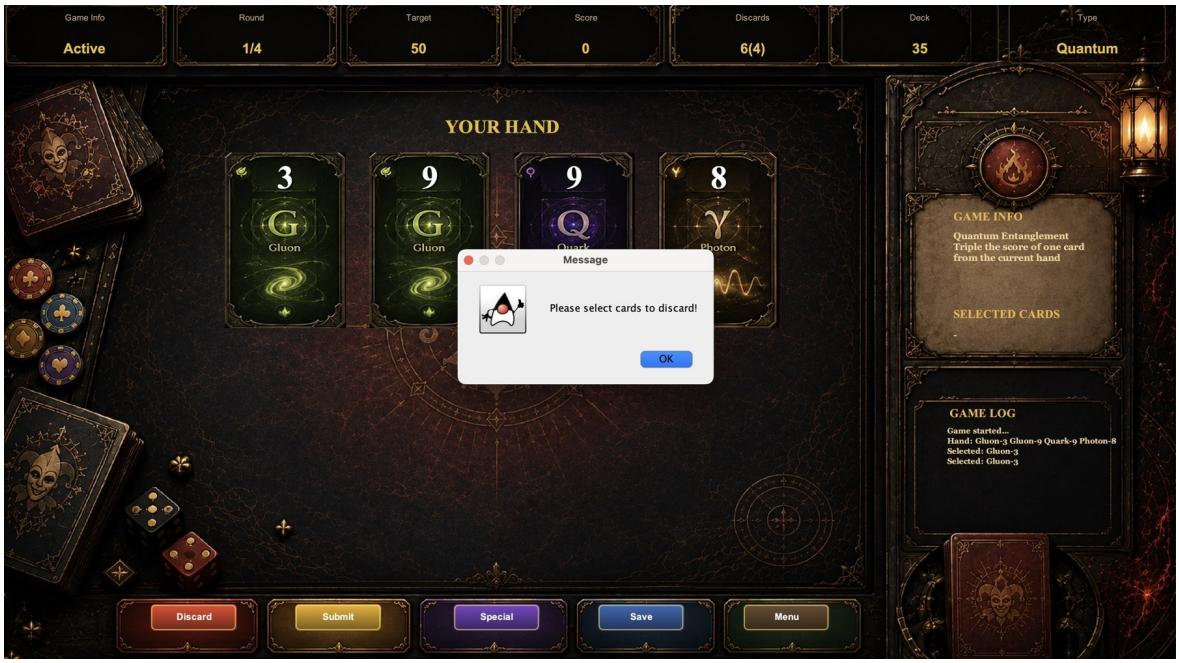

Figure 8. Invalid action feedback shown after pressing Discard without selecting any cards.

3.c. Special Cards

Special cards are used through the Special button or by interacting with a special card in hand. Figure 9 shows the special-card selection dialog, where Transmutation is available. Some special cards mutate the hand immediately, such as Transmutation and Isotope Decay. Others affect scoring, such as Elemental Fusion, Catalyst, Quantum Entanglement, and Superposition.

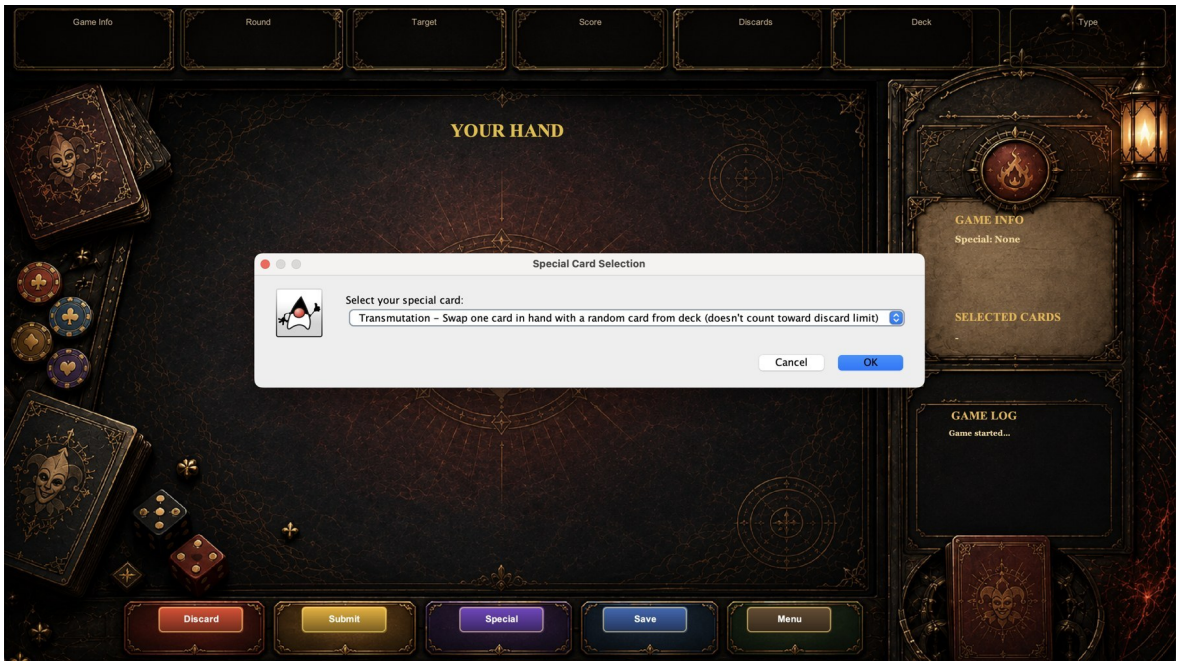

Figure 9. Special Card Selection dialog, showing Transmutation as the available Alchemy special card.

4. Scoring System Demonstration

ScoreManager.scoreCards() is responsible for the scoring decisions. It filters submitted cards to numeric cards, detects the scoring pattern, applies the correct multiplier, and includes active score-changing special-card effects. Four of a kind means four cards of the same type and scores sum x10. One of each type means four different types and scores sum x5. A pair multiplies only the best matching two-card pair by x2; with Catalyst it becomes x3. If no pattern exists, the score is the normal sum.

Case	Rule	Example
Four of a kind	All four cards have the same type; sum x10.	Fire-7, Fire-9, Fire-2, Fire-6 = 240.
One of each type	Four different types; sum x5.	Fire-7, Water-3, Earth-5, Air-2 = 85.
One pair	Best two same-type cards x2; remaining cards normal.	Fire-7, Fire-9, Fire-2, Earth-6 = 40.
No matches	No multiplier.	Fire-7, Water-9, Earth-2 = 18.

Special cards are applied carefully. Quantum Entanglement triples the selected card value before pattern multipliers, so a tripled card inside a pair also benefits from pair scoring. Elemental Fusion treats a hand with at least two same-type cards as four-of-a-kind scoring by multiplying the actual submitted values by 10. Superposition evaluates valid scoring patterns and uses the better result. Philosopher's Stone doubles the current round score once.

5. Round Progression & Game Completion

A full match runs over four turns. When every turn kicks off, beginRound() wipes the current hand, sets aside used cards again, then passes out four new ones. Once someone sends their play via scoreSubmittedHand(), that round's points get recorded, roll into the overall tally, and the following turn loads up. Shown in Figure 10, the ending screen breaks down each round's outcome alongside goal marks, cumulative points, victories counted, personal mean, benchmark average, plus the closing verdict. A score of 195 across rounds gives an average of 48.8 per round. That sits below the medium target benchmark, which runs at 72.5. Even with a victory in Round 3, falling short on average means the outcome counts as a loss. Figure 10 shows how totals decide results.

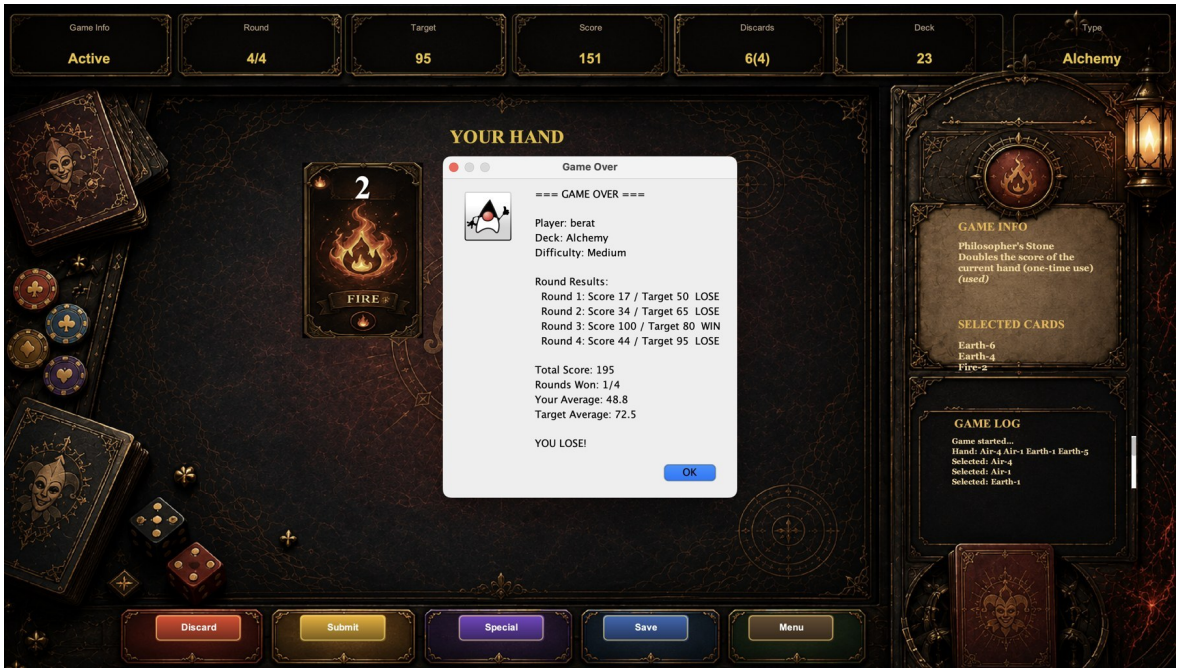

Figure 10. Game Over dialog for an Alchemy game, showing the four round scores, total score 195, player average 48.8, target average 72.5, and final loss.

6. Game State Changes During Play

The active game screen updates player hand, score, deck count, discards, selected cards, round counter, and log messages after each action. Figure 11 shows a loaded Alchemy game at Round 3 with score 53, target 80, deck count 27, and a restored four-card hand. The log panel confirms that the game was loaded and the hand contains four cards.

State	Stored in	Change trigger
Hand	Player.hand	Draw, discard, load, and special-card effects.
Score	Player.totalScore	Round submission and score-modifying specials.
Deck count	AbstractDeck.cards	Each draw removes one card.
Discards	Player.remainingDiscards	Valid numeric-card discard.
Round	Game.currentRoundIndex	Successful hand submission.

Figure 11. Loaded Alchemy game restored at Round 3, with score 53, target 80, four cards in hand, and the log message confirming the loaded session.

7. Save & Load Functionality

7.a. Saving the Game

GameSaveManager.writeGameSave() writes a text save file under the saves folder. Figure 12 shows the generated save file in Finder, and Figure 13 shows the opened save file. The file stores difficulty, deck type, current round, total score, remaining discards, each round score, submitted card counts, current hand, remaining deck, discard pile, selected special card, and special-card usage state.

```
difficulty=Medium
deckType=Alchemy
currentRound=2
totalScore=53
remainingDiscards=6
hand=Earth-4;Earth-2;Water-6;Fire-2;
specialCard=Transmutation
specialCardUsed=false
```

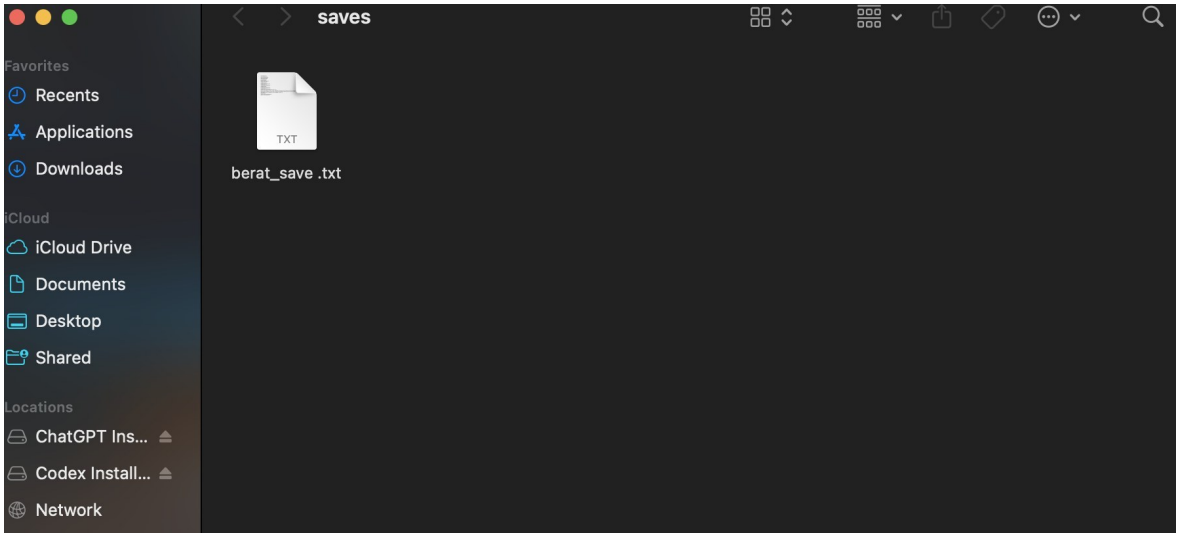

Figure 12. Finder view of the saves folder, showing the generated berat_save.txt file.

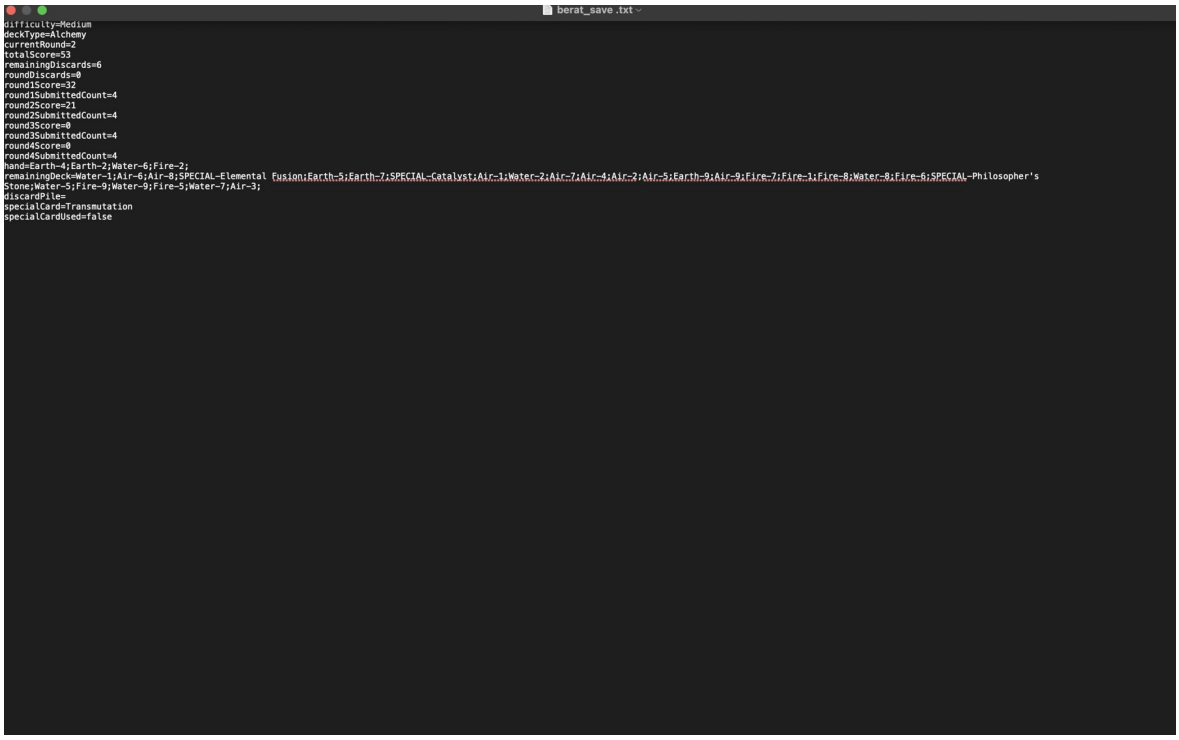

Figure 13. Opened berat_save.txt file, showing the saved deck, round, score, hand, remaining deck, discard pile, and special-card state.

7.b. Loading the Game

Loading reconstructs the same state. GameSaveManager.readGameSave() parses the text file, creates the correct deck through GameEngine.makeDeckForType(), rebuilds the remaining deck and hand, restores round scores and counters, and restores the selected special card. Figure 11 demonstrates the loaded state continuing from the saved file.

8. Logging System

The project uses two kinds of text history. GameLogger appends detailed timestamped events to log.txt, including game start, round start, initial hand, discards, special-card usage, score updates, and game completion. game_history.txt stores completed game summaries for the leaderboard. Figure 14 shows the history file, and Figure 15 shows the same completed result reflected in the menu leaderboard.

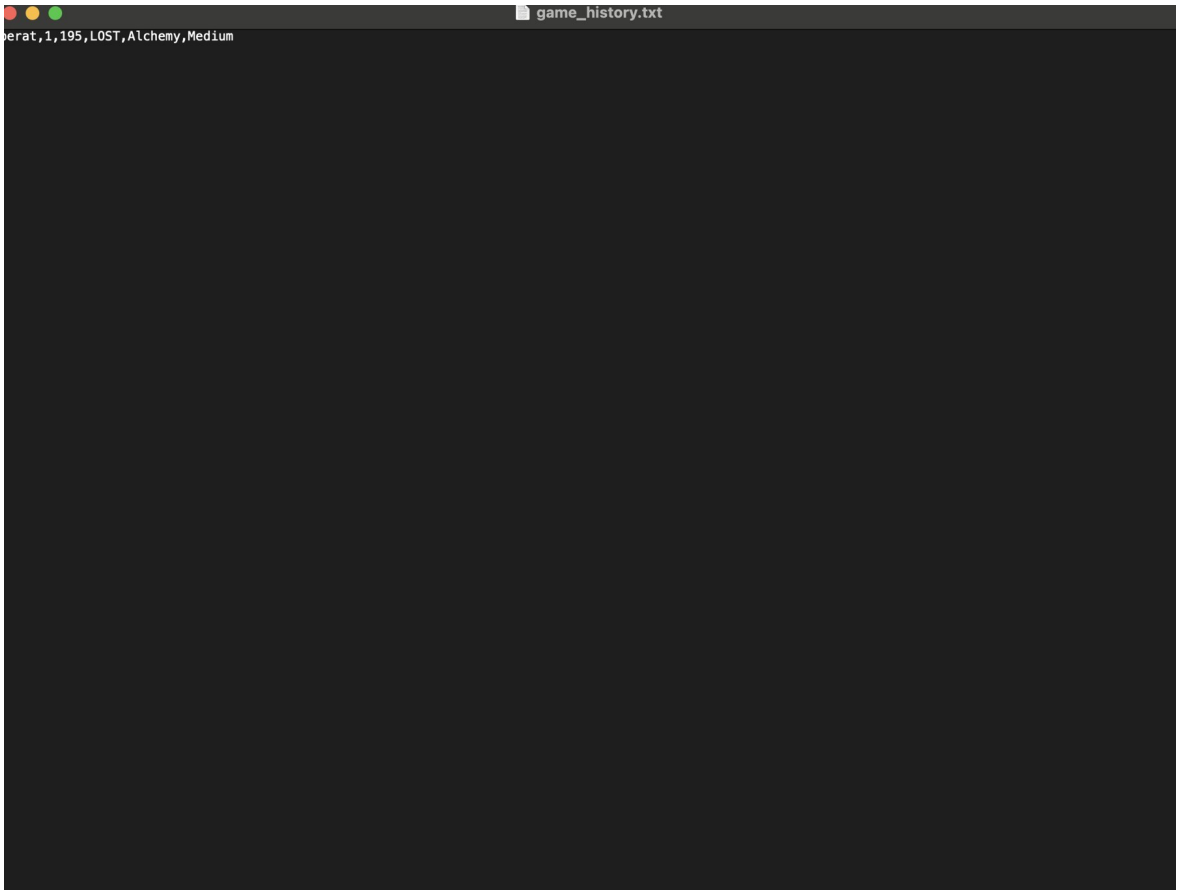

Figure 14. game_history.txt record for the completed game: user berat, session 1, score 195, LOST, Alchemy, Medium.

Figure 15. Main menu leaderboard after the completed Alchemy game, showing berat with score 195 and status LOST.

9. Error Handling Demonstration

Wrong moves get caught two ways - through checks inside the code and alerts on screen. When it comes to tossing cards, Player.discardFromHand says no by returning false if the card is protected, already spent, or simply not held. Trying to score a hand? Game.scoreSubmittedHand blocks anything blank or too large. Loading trouble shows up when GameSaveManager raises an exception for missing files. Pop-up notes in the interface explain what went wrong, like that alert about discarding seen in Figure 8.

10. Visual and Functional Differences Across Games

Look at Figures 5, 6, yet 7 - they show how Alchemy, Element, along with Quantum differ in look and play. Score boosts plus combining types drive Alchemy forward. With Element, getting cards back matters just as much as shifting their kind or worth. Then there is Quantum - focused on planning moves ahead, seeing what comes next, joining pieces smartly, chasing high-value arrangements. Even so, every version follows identical steps each turn: pull a set of cards, pick which ones to keep, maybe drop some aside, perhaps trigger a unique effect, send your choice in, check if points meet the goal, repeat this three more times without change.

Part 2

Project Design Description:

The project is organized into model classes, deck classes, GUI classes, engine classes, and save/load support classes. The design uses inheritance for cards and decks, composition for game state, and Swing event listeners for user interaction.

11. Class Relations

Game owns four Round objects and tracks target scores. Player owns the deck, hand, discard pile, selected special card, discard counters, and total score. Round stores one submitted Hand and uses ScoreManager.scoreCards() to calculate the score. AbstractDeck owns the card list and draw behavior, while AlchemyDeck, ElementDeck, and QuantumDeck fill that list with deck-specific cards.

Class/package	Responsibility
Card, NumericCard, SpecialCard	Base card hierarchy.
AbstractDeck and deck subclasses	Deck creation, shuffle, draw, and remaining-card count.
Player	Hand, deck, discards, selected special, and total score.
Game and Round	Round progression, targets, submission, and final result.
ScoreManager	Pattern detection and special-card scoring.
GUI package	Login, menu, rules, and gameplay windows.
saveload package	players.txt, game_history.txt, saves, and log.txt.

12. Inheritance, Type Hierarchies, Interfaces, and Abstract Classes

Card is abstract because every card must be either numeric or special. NumericCard extends Card and adds value and lock state. SpecialCard extends Card and adds description, usage state,

and applyEffect(Player player). AbstractDeck is also abstract: it provides shared draw and shuffle logic, while concrete deck classes define their own numeric and special cards.

```
public abstract class Card { protected String type; }
public class NumericCard extends Card { private int value; }
public abstract class SpecialCard extends Card {
    protected String description;
    public abstract void applyEffect(Player player);
}
```

13. GUI Components

LoginFrame handles authentication and registration. MainMenuFrame handles new games, load games, deck selection, logout, and leaderboard display. RulesFrame displays the How to Play information. GameFrame is the main gameplay window: it draws card components, highlights selected cards, updates HUD labels, opens special-card dialogs, displays round results, and routes user actions to Game and Player.

14. .txt File Processing Details

players.txt stores account data as comma-separated username and password lines; Figure 16 shows this file. game_history.txt stores completed game records for the leaderboard. Save files use key=value lines because they are readable and simple to parse. Cards are stored as text tokens such as Earth-4 or SPECIAL-Catalyst, then reconstructed during loading.

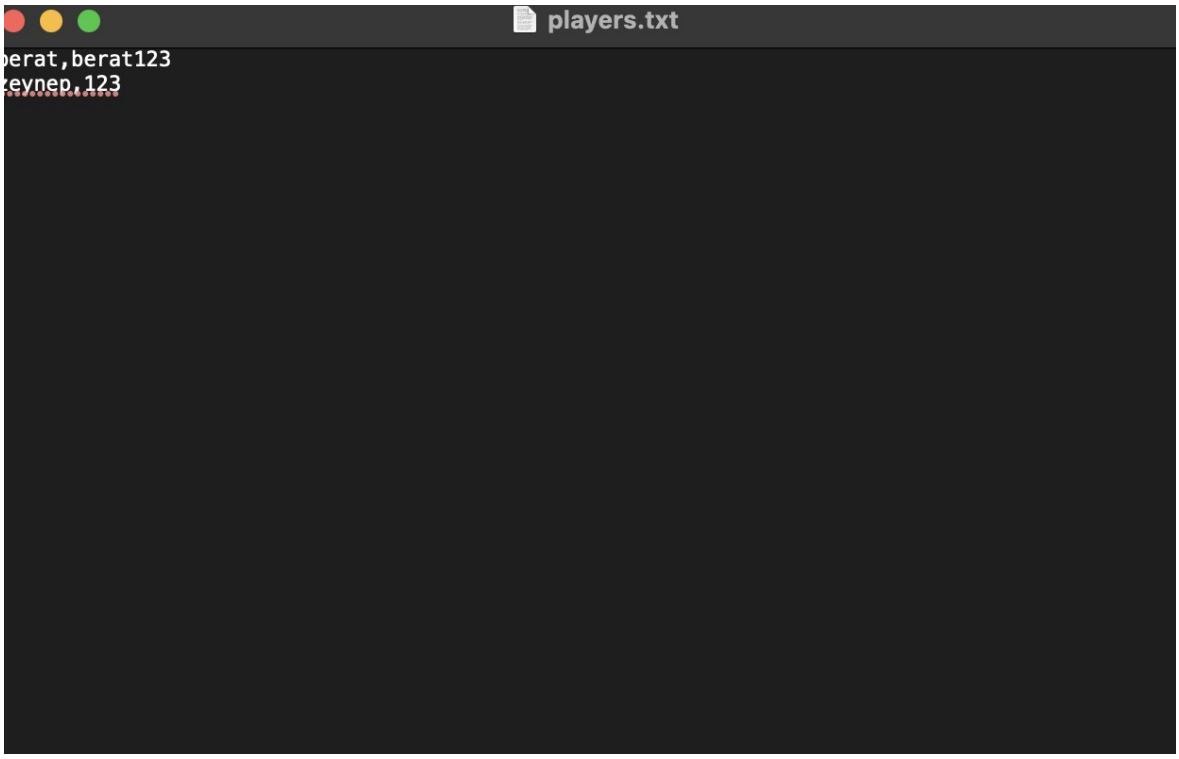

Figure 16. players.txt opened in a text editor, showing registered users as comma-separated username and password records.

15. Implementation Details

The most important implementation detail is that scoring is centralized in ScoreManager.scoreCards() instead of being spread across GUI code. This keeps scoring consistent for normal submission, special-card previews, and loaded games. GameFrame manages user events with methods such as submitPickedCards(), discardPickedCards(), and refreshGameScreen(), but ScoreManager decides the numeric result. GameEngine creates the correct deck and game objects from menu selections.

16. Exception Handling

The project combines exceptions and defensive returns. File operations throw or catch IOException and FileNotFoundException. Invalid model actions return false or zero when appropriate. GUI code then shows feedback dialogs so the player understands the problem. This is visible in Figure 8, where the game tells the player to select cards before discarding.

17. Additional Design Details

The project separates persistent account data, playable save data, and completed-game history. This separation keeps login, save/load, and leaderboard responsibilities independent. Visual assets are loaded from project resources, which makes export safer because the program does not depend on a developer-specific absolute path.

18. Detailed Scoring Rule Walkthrough

Scoring became the core piece shaped by rules in Kulatro - that is why the description here stays thorough. Not just summing up what numbers show on cards decides points. First thing checked? Which pattern the played hand matches. Then comes its specific multiplier. Only once those are set do effects from special cards step in, if they change anything at all. Tucking all this into ScoreManager.scoreCards() helps testing. Plus it stops the interface from quietly rewriting how scoring should work.

A single hand might hold up to four cards, sometimes fewer. When special ones show up, they don’t count toward points - only number cards do. That’s on purpose. Actions or changes come from specials; real score bits come from numerics. So ScoreManager pulls out just the numbered ones first, then looks for point patterns. Zero shows up when numbers aren’t part of the cards played. That rule blocks wrong entries, making sure scores stay clear every time.

Top scoring regular hand? Four of a kind. In Kulatro, that means four number cards sharing one element, not the same numbers. Take Fire-7, Fire-9, Fire-2, Fire-6 - same element, counts as four of a kind. Add those values: total is 24. Multiply by ten for the set bonus, lands at 240 points. Focusing on one element lifts your score. Matching elements become powerful choices during play.

One way to hit a big score involves picking just one card per element. Four number cards must appear, each from a separate kind. Take these: Fire-7, Water-3, Earth-5, Air-2 - found in the Alchemy set. Added together, they make 17. When that total meets the one-per-type bonus of five times, it jumps to 85. One way it works is by valuing variety instead of stacking similar things. What happens next depends on the mix, since every build carries four distinct types while sharing a common scoring backbone. The pair case is more subtle than the other two cases. A pair is formed by two cards of the same type. The current implementation chooses the best

possible two-card pair from the submitted hand, multiplies only that pair, and adds all other numeric cards normally. This means that if a submitted hand contains Fire-7, Fire-9, Fire-2, and Earth-6, the best pair is Fire-9 plus Fire-7. The pair part is (9 + 7) x 2 = 32, and the remaining cards add 2 + 6, so the final score is 40. This avoids accidentally treating three cards of the same type as if all three were multiplied by the pair rule.

If no pattern is found, the score is just the normal sum of the submitted numeric values. For example, Fire-7, Water-9, and Earth-2 do not make four of a kind, do not make one of each type, and do not make a pair. Therefore, the score is 18. This fallback makes partial hands usable while still encouraging the player to search for stronger patterns.

Rule	Condition	Multiplier	Design reason
Four of a kind	Four numeric cards, all same type.	x10 on full submitted value sum.	Strong reward for type concentration.
One of each type	Four numeric cards, all different types.	x5 on full submitted value sum.	Reward for balanced deck coverage.
One pair	At least two cards share a type.	x2 on best pair only; rest normal.	Reward matching while preventing overcounting.
No matches	No valid scoring pattern.	No multiplier.	Keeps every legal submission scorable.

Special cards are then layered on top of this pattern system. The important implementation detail is that score-changing special cards are not all handled in the same way. Some modify card values before multipliers, some modify the multiplier itself, some transform the pattern that the hand is allowed to count as, and some change the hand before scoring. This is why the report separates scoring cases from special-card behavior.

One way to adjust scores involves Quantum Entanglement. This effect multiplies a single number card in your hand by three. When that card forms a pair, its new total counts toward the match. Suppose someone plays Gluon-9 with Gluon-5 as a set. Using Quantum Entanglement on Gluon-5 changes the numbers seen - now it reads nine and fifteen instead. Twelve times two gives twenty-four, yet when doubled later it hits forty-eight. That twist means Quantum Entanglement steps in early - before the doubling - not once everything adds up at the end.

What sets Catalyst apart? It changes how much each pair counts. Most times, pairs double your score. When Catalyst's turned on, that boost jumps to triple instead. After scoring the pair, everything else gets totaled like usual. Take Fire-7, Fire-9, Fire-2, Earth-6 - these usually total 40. When Catalyst kicks in, Fire-9 paired with Fire-7 climbs to (9 + 7) times 3, landing at 48. Then 2 joins 6 on the side, pushing everything up to 56.

When Elemental Fusion kicks in, how hands are scored shifts. Having two matching types means the whole set counts as four identical ones for points. It doesn’t pretend there are more cards than played; only real numbers matter, then multiplied like a four-of-a-kind. So instead of picking Earth-1 and Earth-5 to reach 17, the total first adds up to 11 - then jumps to 110. That difference sticks close to what the card says, using just what shows on the table.

One way to look at Superposition: it picks between two scoring options and keeps the higher one. Instead of guessing, the system checks every legal way to score the numbers played, then uses the top outcome. Sometimes that number matches what you’d get anyway - normal scoring often finds the best fit on its own. What changes here? The choice becomes clear instead of hidden. When more than one pattern fits, there's no risk of settling for less. It just works silently, ensuring nothing gets overlooked.

Special card	Score effect	When it matters most
Quantum Entanglement	Triples one selected card value before the pattern multiplier.	Best when the selected card is inside a pair or high multiplier pattern.
Catalyst	Raises pair multiplier from x2 to x3.	Best when the hand contains a strong high-value pair.

Elemental Fusion	Treats a matching-type hand as four-of-a-kind scoring.	Best when at least two same-type cards are already present.
Superposition	Evaluates valid scoring patterns and uses the stronger result.	Best in ambiguous hands with multiple possible scoring interpretations.
Noble Gas	Locked card value counts twice for scoring.	Best when locking a high-value card that will remain useful.
Electron Bond	Makes two different types count as a matching pair.	Best when the hand has high values but no natural pair.
Philosopher's Stone	Doubles the current round score once.	Best after creating a high-scoring hand.

The game-over example in the demonstration gives a complete scoring context. In the shown game, the player finished with round scores of 17, 34, 100, and 44. The total score is 195. The player average is 195 divided by 4, which is 48.75 and displayed as 48.8. Medium difficulty has targets 50, 65, 80, and 95, so the target average is 72.5. Because 48.8 is lower than 72.5, the final result is a loss even though Round 3 was won. This demonstrates that the final game result is based on average performance across all four rounds, not only on the number of individual rounds won.

19. Detailed Special Card Behavior

Step by step, these decks grow from the same base setup - yet twist into new paths thanks to different cards steering each one. Watch carefully and small differences reveal code-like logic running underneath. Rather than listing effects, we examine how moves connect to programming ideas and real-time decisions. Subtle links tie purpose to experience, so form shapes more than rules ever could.

Of every deck available, Alchemy piles up points quicker than the rest. Once your setup feels solid, playing Philosopher’s Stone boosts whatever hand you’re holding - pulling it at the right

moment makes all the difference. With Transmutation, trade out any single card for a fresh mystery draw; this move won’t eat into your discard limit. Hold duplicates - two or more of a kind - then watch Elemental Fusion turn them into four-of-a-kind combos worth bonus points. A sudden surge lifts the pair value, thanks to Catalyst's amplified multiplier. With speed now on its side, Alchemy rises as the top choice whenever points need to pile up fast.

Start by using Element to steer how you manage pieces. Hit Periodic Boost and watch digits in your grasp climb higher - each one pushing totals upward. Rather than trade cards out, Noble Gas freezes just one, then doubles its face value repeatedly as rounds pass. When luck dips early, Isotope Decay clears whatever you hold, handing four new picks without following standard discard steps. Out of nowhere, progress kicks in quick. If odd parts carry hidden strength, Electron Bond pulls them close - suddenly they fit. Stalled moments fade when the deck adapts quietly, making awkward arrangements work differently than expected. Poor starts breathe slower, then reshape themselves into what matters.

Most players find quantum moves easiest to plan around. Pulling a card triggers Quantum Entanglement, tripling its worth - success leans on moment, not chance. Rather than risk a blind draw, Superposition checks both potential outcomes and holds the better one, useful when paths blur. Another way in: Gluon Bind merges two numbers into one capped at nine, then brings a fresh card forward - reshaping hands quietly. What you keep shifts as the board changes. One move at a time shifts things slightly, without noise. From silence, Photon Burst reveals glimpses - three cards exposed - for just an instant to swap one against your hand. Now options come clearer, more defined, since sight stretches forward. Where others stumble unseen, their paths fuzzier, less aimed.

Deck	Special card	Implementation category	Result in gameplay
Alchemy	Philosopher's Stone	Post-score multiplier	Doubles the submitted round score once.
Alchemy	Transmutation	Hand mutation	Swaps one hand card with a random deck card.

Alchemy	Elemental Fusion	Pattern override	Uses four-of-a-kind scoring if at least one pair exists.
Alchemy	Catalyst	Multiplier modifier	Raises pair scoring from x2 to x3.
Element	Periodic Boost	Value mutation	Adds +2 to numeric cards in hand.
Element	Noble Gas	Persistent value modifier	Locked card remains and counts twice.
Element	Isotope Decay	Hand reset	Replaces the entire hand without using discard limit.
Element	Electron Bond	Type transformation	Allows different types to count as a pair.
Quantum	Quantum Entanglement	Value multiplier	Triples one selected card.
Quantum	Superposition	Pattern comparison	Uses the better valid scoring result.
Quantum	Gluon Bind	Merge and draw	Combines two values up to 9 and draws a replacement.
Quantum	Photon Burst	Deck preview and swap	Reveals upcoming cards and swaps one into hand.

A key design choice is that all special cards inherit from SpecialCard. This means the GUI can treat them as cards for display purposes, while the game logic can identify the concrete subclass when applying the effect. This gives the project a clear extension point. If a fourth deck were added later, new special cards could be created by extending SpecialCard and implementing applyEffect(), without rewriting the general card hierarchy.

20. Detailed Gameplay Flow

Starting off, the person picks login or sign up. When they type details, LoginFrame talks to PlayerManager to compare what was entered. That manager handles reading from and saving to a text file for users. Only when everything matches right does the system show MainMenuFrame next. From the start, this menu does more than move you around. It holds live details like your overall score and who's on top. What others scored sticks around because past rounds shape the rankings shown here. Before jumping into a new round, you see how things stood last time.

Starting fresh? The first thing that pops up is a request for a session name. That title sticks - it shapes how the saved game shows up later. Picking what to play comes next - deck choice sets the scene: cards change, looks shift, even unique ones appear based on it. Difficulty tags along here too, picked at the same time. Whatever difficulty you pick sets the goal numbers. Once those choices lock in, the system builds a matching game setup along with its cards. With everything ready, gameplay kicks off inside the main window.

Four cards sit in the hand when each round kicks off. Round number up top, along with how many points you need, your current total, whether discards are allowed, how many cards remain, and what kind of deck it is - all laid out across the header. Off to the side runs a strip showing whatever power card is live right now, which ones youve picked, plus a running note of actions taken so far. Because everything stays on screen - target, leftover deck, active boost - you never have to dig through menus. Seeing it all together means choices come faster, smoother, without hunting around.

Picking a card isn’t permanent. One tap marks it, shifts how it looks, also places it into the Selected Cards group. Tap once more - it unmarks just like that. Trying out choices becomes easy, stress free. Four cards max in play since scoring only works with hands that size. When wrong moves pop up, the system blocks them quietly, keeping everything stable even if the interface stumbles. Game stays clean no matter what glitches show.

One way to proceed involves discarding. Choosing numbers happens before hitting Discard. When picks fit rules and discard count permits, that card shifts to the discard stack, another gets pulled in its place, counts adjust accordingly. Pressing Discard while nothing's picked brings up a note on screen. That incorrect move appears in a snapshot showing text telling players to choose cards first.

One way to play involves special cards. From a popup menu, some get triggered. Others come straight out of your hand. When it matters, the interface checks what you want. That step exists since certain outcomes can't be undone later in the round. Picking Transmutation swaps one card. Choosing Isotope Decay replaces every card held. Pauses like these help prevent unintended moves.Submitting the hand ends the current round. GameFrame sends the selected cards to Game.scoreSubmittedHand(), which creates a Hand, sends it to the current Round, and passes the chosen special card to ScoreManager.scoreCards(). The round stores the result, the player total score is updated, and the round index advances. The next round starts unless four rounds have already been completed. After the final round, the game displays the game-over dialog with per-round results, total score, average score, target average, and final win/loss result.

21. Detailed Class Design

Five levels make up the structure. At the start, cards form the base - these include Card, along with NumericCard and also SpecialCard. Following that comes the deck setup, built around AbstractDeck together with three specific implementations below it. Next appears the stage where actions unfold: entities like Player, Hand, and Round, joined by ScoreManager plus Game. Each piece fits within a broader sequence, none standing apart from the whole. Fourth comes the interface tier - LoginFrame, MainMenuFrame, followed by RulesFrame and then GameFrame. Below that sits storage and auxiliary functions: PlayerManager handles user data while GameSaveManager stores progress, SpecialCardLoader pulls unique cards, GameLogger records events, alongside CardImageLoader managing visuals.

Though simple in form, the card exists as an abstraction since the design avoids forming cards without defined roles. Each one belongs strictly to NumericCard or SpecialCard. While NumericCard brings a number along with a lock indicator - needed by Noble Gas - SpecialCard contributes a written note, status tracker, and a function called applyEffect(Player player). The shared trait across both is their kind. Yet how they score differs entirely from how they act. Structure stays clear through this split.

Despite common structure, variation begins at setup. Storage and draw mechanics apply universally. Shared by all implementations, the list of cards resides in parent level code. Shuffling method stays consistent throughout subtypes. Count tracking follows same pattern everywhere. What changes is determined during creation phase. Specific values and unique

elements depend on subclass choice. Uniform retrieval process remains unaffected. Distinct composition emerges without altering core logic. Behavior repeats identically regardless of content type.

Most control over changing game data rests with Player. The active deck, cards held, pile set aside, chosen special card, overall points, discards left, and those allowed each round belong to it. Communication often flows through Player when GameFrame updates what appears on screen. Verification happens here too - before any discard proceeds, checks confirm type, quantity, and whether the card is actually in hand. Rules around discarding stay enforced because of these validations.

A single scoring attempt defines what a Round is. With its number, intended score, cards played, result, and whether it finished, structure emerges naturally. Ownership of four Rounds by a Game allows averaging at the end without extra steps. Stored data stays clear since every round's outcome records individually. Rebuilding saved sessions becomes straightforward when each entry stands alone.

From the player’s perspective, ScoreManager holds no internal status. Given a set of cards along with one marked card, it computes a value then delivers the result. Its role remains limited to evaluating scores, avoiding control over gameplay itself. Because of this separation, understanding its function becomes more straightforward. Duplicate logic in interface previews and end-stage processing is less likely to emerge. Ownership of game progression stays elsewhere by design.

Layer	Main classes	Purpose
Card model	Card, NumericCard, SpecialCard	Defines the common card hierarchy and separates scoring cards from action cards.
Deck model	AbstractDeck, AlchemyDeck, ElementDeck, QuantumDeck	Creates, shuffles, and draws deck-specific cards.
Gameplay model	Player, Hand, Round, ScoreManager, Game	Stores active state, validates actions, calculates scores, and controls round

		progression.
GUI layer	LoginFrame, MainMenuFrame, RulesFrame, GameFrame	Displays the application and handles user events.
Persistence/support	PlayerManager, GameSaveManager, GameLogger, SpecialCardLoader, CardImageLoader	Stores players, saves games, logs events, loads metadata, and resolves images.

22. Detailed File Processing

Using text files allows clear visibility when documenting code behavior within reports. Because simplicity matters, players.txt holds user data through commas on separate lines. One entry per person appears in the example shown. Entries like berat,berat123 and zeynep,123 reflect how login details are stored plainly. Storage remains readable without complex tools due to this method. Basic persistence emerges naturally from such straightforward design.

Inside the system, game_history.txt holds past match outcomes. Each line - like berat,1,195,LOST,Alchemy,Medium - lists player ID, round label, points earned, outcome status, card set used, and challenge level. These entries get pulled by the ranking display when users return to the home interface. Rather than vanishing after play ends, results appear both in exit messages and stored rankings. From there, they remain visible each time someone opens the app. One record forms once a session finishes successfully. Data flows silently behind scenes, linking performance summaries with navigation views. Once written, an entry stays unless manually cleared. This file acts as a bridge across different parts of the program. Even small actions during matches contribute to long-term logs. Later visits show what happened earlier without extra steps needed. The process runs automatically, requiring no input beyond normal play. Results build up gradually over repeated sessions. A single structure supports multiple features at once. Information moves from temporary screens into lasting storage. No separate save step exists; completion triggers recording directly. Behind simplicity lies coordination between functions that seem unrelated. Every comma-separated value has purpose within larger mechanics. What appears minimal supports continuity across uses. Consistent formatting allows reliable interpretation every time it loads. Without this log, progress would reset entirely on restart.

Not merely outcomes, yet full conditions belong within save records. Where history logs settle on conclusions alone, saving demands recreation of active play. Because of this requirement, elements such as difficulty setting appear alongside deck classification. The present round stage matters, just like accumulated points across turns. Discard limits still available are noted, along with marks earned each round. How often submissions occurred gets tracked too. Cards held at pause stay recorded, as do those unseen in the stack. Even discarded options remain listed. A chosen unique ability finds its place, including if it has already activated. That depth accounts for increased size compared to leaner historical notes.

One reason the save format relies on key=value pairs lies in their clarity and straightforward interpretation. When data loads, GameSaveManager processes every line by examining what comes before the equal symbol. Take deckType= at the start of a line - whatever follows sets the kind of deck used. Lines beginning with currentRound= have their trailing characters converted into whole numbers instead. Separated by semicolons, the card lists become either NumericCard or SpecialCard once again. Though basic, this method of serialization works without complication.

File	Example content	Purpose
players.txt	berat,berat123	Stores registered login data.
game_history.txt	berat,1,195,LOST,Alchemy,Medium	Stores completed game summaries for leaderboard display.
saves/<user>_<session>.txt	currentRound=2, totalScore=53, hand=Earth-4;Earth-2;Water-6;Fire-2;	Stores full resumable game state.
log.txt	[timestamp] ROUND 1 START - Target Score: 50	Stores chronological gameplay events for debugging and demonstration.
cards/*.txt	Special-card names and descriptions	Stores or regenerates special-

		card metadata.

23. GUI Design and Event Handling Details

Clicks, taps, or typed entries set things in motion. Rather than follow a fixed sequence, the interface responds when users do something noticeable. A press on any key button triggers code tied directly to it. That link might change what appears on screen or hand off work to behind-the-scenes logic. Since players pick one move at a time during play, this fits how they naturally interact. Not by entering lines of instruction, but through isolated decisions made moment by moment.

Starting off, LoginFrame handles sign-ins. Users type their name and code here before picking one of three choices - enter, create account, or learn gameplay rules. Moving through, the next screen manages what happens during a visit. This part shows who is active, points earned overall, rankings, along with links to move around. At each turn, GameFrame highlights choices while displaying every active element. Meanwhile, RulesFrame offers clarity through context - never altering what is already in motion.

The GameFrame stands out as the trickiest window, handling coordination across several model changes. A click on any card triggers toggleCardPick(), which alters selectedCards while refreshSelectedCardsText() adjusts the side panel's display. Clicking Discard activates discardPickedCards(), verifying what’s chosen before passing control to Player.discardFromHand() for execution. Pressing Submit runs submitPickedCards(), routing the current hand into Game.scoreSubmittedHand() for processing. Clicking Special triggers useSpecialCardNow(), which either shows a popup or activates the selected power. Once that finishes, refreshGameScreen() updates counters and redraws cards in hand.

When a player tries to discard without picking a card, nothing happens on screen. That could feel odd, like the button broke. So instead, a small message appears right away. It says selection comes first, then discard. This way, people know what went wrong. Feedback shows up where the mistake occurs - right in front of them. Rules about input checks live inside the visuals now, not just behind scenes. Confusion drops when clarity steps in mid-click. The system speaks up before silence causes doubt.

24. Testing and Verification Notes

During development, manual checks covered several scoring instances due to the difficulty of spotting errors in a visual format. Since missteps can occur when three matching cards appear, verification confirmed only the strongest two-card pair gains the multiplier. In situations involving dual potential pairs, accuracy relied on consistent selection rules. Before applying pair bonuses, validation ensured the chosen card value was first tripled under Quantum Entanglement. For Elemental Fusion, testing focused on whether real input data drove outcomes instead of fabricated duplicates.

Another method involved a simple step-by-step model to run through several hands and check approximate totals. Not meant as a substitute for complete code tests, it served more as a consistency check on typical results. From ten trial sets, the sum reached 588, yielding 58.80 when divided equally among them. Such outcomes gave confidence - scores for common patterns, pairs, and enhanced cases aligned loosely with what the Medium benchmark suggested.

Beginning with visual confirmation, the process of saving and loading advanced via image evidence. Shown within the file: currentRound set to 2, totalScore at 53, discards left numbering 6, individual round tallies present, along with hand composition, deck remainders, discarded items listed, active special card marked, its usability indicated. Following reload, display shifts - now in Round 3, score unchanged at fifty-three, category reads Alchemy, hand holds exactly four playing pieces, screen logs note appearance confirming retrieval. State restoration becomes clear only after observing these elements align precisely across both moments.

Verification occurred after one full playthrough ended at main screen. Nineteen-five appeared as final tally within defeat message. Same number and outcome recorded inside game_history.txt file image. Player entry listed on ranking view: 195 points, marked LOST. Data continuity confirmed - runtime score entered stored log, later reappeared in interface.

25. Limitations and Future Improvements

Completion of the present task meets assigned criteria, yet expansion could allow enhancements. Stored credentials appear as raw characters within players.txt. Real-world systems demand cryptographic hashing with unique salts instead of clear-text entries. Within academic scope,

unencrypted values serve purpose by clarifying data handling procedures. Such practice remains unsuitable when deploying authentication mechanisms beyond classroom settings.

Next comes the fact that the save file parser requires precise key labels and specific line layouts. While suitable within tightly managed conditions, an alternative using JSON or similar structured data methods might offer better reliability. Such formats tend to lower mistake rates during interpretation while simplifying checks for layered groupings - hand contents, deck arrangements, discarded cards, round outcomes. What exists now relies on key=value pairs due mainly to clarity and alignment with learning aims focused on handling plain text files.

One possibility involves splitting certain GUI elements into more compact units. Although GameFrame now manages rendering, picking mechanics, pop-up interfaces for unique cards, score transmission, along with screen updates, expansion might justify change. In broader implementations, the area showing player hands, heads-up display segment, history tracker, and distinct card interface may exist as individual modules. Such division would shrink GameFrame while improving testing clarity per unit. Yet given current scope, unified structure supports clearer tracing from interaction through state adjustment. Structure remains intact here due to scale considerations.

Last of all, automation might grow within ScoreManager and GameSaveManager. Because scoring follows strict rules, testing each pattern along with special card mixes brings clear benefit. Instead of skipping checks, one approach runs through creating a game, saving it, then reloading to inspect key values. With such layers in place, future edits - say, adding fresh decks or unique cards - carry less risk.

References

[1] COMP132 Advanced Programming Project Report format and demonstration requirements, screenshots provided in the project brief, Spring 2026.

[2] Oracle. Trail: Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/index.html

[3] Oracle. Java SE 21 API documentation for javax.swing. https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/package-summary.html

[4] Oracle. Java SE 21 API documentation for BufferedReader. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/BufferedReader.html

[5] Oracle. Java SE 21 API documentation for Java List. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/List.html

[6] OpenAI ChatGPT. AI-assisted visual asset generation for Kulatro card/background images, May 2026.

[7] Google Gemini. AI-assisted visual asset generation for Kulatro card/background images, May 2026.